

OPERATION IRON RAVEN

Penetration Test Risk Register

Prepared by	Daniyal Ahmed	Registration	C11/26/EHIT/17331
Course	CIP-A105 — Offensive Security Operations II	Assessment	CTF 1 — Operation Iron Raven
Date	16–17 September 2026	Classification	Training Use Only
Risk Method	Likelihood × Impact (1–10 each)	Max Score	100

Critical

High

Medium

Low

FIND-01 — Insecure Directory Listing — /vendor/ Directory Exposed

Affected Asset / Service	Category	Severity	Risk Score
Apache HTTP — /vendor/ path (port 80)	Misconfiguration	Medium	40/100 (L:8 × I:5)

Technical Description

Apache directory listing was enabled on /vendor/. Any unauthenticated visitor could browse and read PHPMailer library files including the VERSION file, which disclosed the exact library version (5.2.16) and directly enabled the critical exploitation in FIND-02.

Recommended Control

Add "Options -Indexes" to the Apache VirtualHost or .htaccess. Move vendor libraries above the web root or restrict access with "Require all denied" in the directory block.

Evidence: E-011, E-012, E-013, E-014

FIND-02 — PHPMailer 5.2.16 — Remote Code Execution (CVE-2016-10033)

Affected Asset / Service	Category	Severity	Risk Score
PHPMailer library — contact.php form on port 80	Unpatched Vulnerability	Critical	100/100 (L:10 × I:10)

Technical Description

PHPMailer versions below 5.2.18 are vulnerable to CVE-2016-10033. The vulnerability allows unauthenticated RCE by injecting arbitrary sendmail arguments via the Sender/From field of the mail function. Exploitation via EDB-ID 40974 delivered a PHP reverse shell through the contact form,

granting an interactive shell as www-data without any credentials.

Recommended Control

Upgrade PHPMailer to 5.2.27 or later immediately. Sanitise and whitelist all mail sender inputs server-side. Deploy a WAF rule to detect sendmail argument injection patterns.

Evidence: E-014, E-015, E-016, E-017, E-019, E-020

FIND-03 — Cleartext Database Credentials in wp-config.php

Affected Asset / Service	Category	Severity	Risk Score
/var/www/html/wordpress/wp-config.php	Credential Exposure	High	81/100 (L:9 × I:9)

Technical Description

The MySQL database username (root) and password were stored in plaintext using define() statements in wp-config.php. Any process with filesystem read access — including the www-data web shell obtained via FIND-02 — could harvest full database administrative credentials in seconds.

Recommended Control

Migrate credentials to environment variables or a secrets manager. Create a dedicated, least-privilege MySQL user for WordPress only — never use the root account for application database connections.

Evidence: E-023, E-024

FIND-04 — WordPress User Password Hashes Exposed via Database

Affected Asset / Service	Category	Severity	Risk Score
MySQL — wordpress.wp_users table	Data Exposure	High	64/100 (L:8 × I:8)

Technical Description

With MySQL root access, the operator queried wp_users and extracted phpass password hashes for all registered accounts (michael and steven). These hashes are vulnerable to offline dictionary attacks. Successful cracking would enable WordPress admin access and potential credential reuse on SSH.

Recommended Control

Restrict the WordPress DB user to DML on the wordpress database only — no SELECT on administrative tables from untrusted users. Enable MySQL audit logging. Enforce strong, unique passwords for all accounts.

Evidence: E-026, E-027

FIND-05 — MySQL Service Running as OS root

Affected Asset / Service	Category	Severity	Risk Score
MySQL 5.5.60 daemon — localhost:3306	Privilege Misconfiguration	Critical	100/100 (L:10 × I:10)

Technical Description

The mysqld process ran with full operating system root privileges — a severe misconfiguration. Any SQL-level exploit against MySQL, including User-Defined Function injection (FIND-06), executes with OS root authority. This collapses the boundary between database and system compromise entirely.

Recommended Control

Configure mysqld to run as the dedicated "mysql" system user. Set user=mysql in /etc/mysql/my.cnf and restart. Verify with: ps aux | grep mysqld — should show mysql, not root.

Evidence: E-025, E-036, E-038, E-041

FIND-06 — MySQL UDF Injection — Arbitrary OS Command Execution

Affected Asset / Service	Category	Severity	Risk Score
MySQL plugin directory — /usr/lib/mysql/plugin/	Exploitation Technique	Critical	90/100 (L:9 × I:10)

Technical Description

Using MySQL LOAD_FILE and DUMPFILE, a malicious shared library (1518.so) was written to the MySQL plugin directory and registered as the do_system() UDF. This function executed arbitrary OS commands at the mysqld privilege level (root), enabling direct system manipulation from within SQL queries.

Recommended Control

Set secure_file_priv = "/var/lib/mysql-files" in mysqld.conf to prevent DUMPFILE writes to system directories. Revoke the FILE privilege from all non-administrative MySQL accounts. Implement file integrity monitoring on the plugin directory.

Evidence: E-035, E-036, E-037, E-038

FIND-07 — SUID find Binary — Root Shell Privilege Escalation

Affected Asset / Service	Category	Severity	Risk Score
/usr/bin/find (SUID bit set via FIND-06)	Privilege Escalation	Critical	100/100 (L:10 × I:10)

Technical Description

The do_system() UDF set the SUID bit on /usr/bin/find. Any local user could then execute "find <file> -exec /bin/sh \;" to spawn a root shell, as the SUID binary inherits root ownership at execution time. This was the final step yielding complete OS root compromise.

Recommended Control

Immediately run: `chmod u-s /usr/bin/find`. Audit all SUID/SGID binaries regularly: `find / -perm -4000 -o -perm -2000 2>/dev/null`. Resolve root causes in FIND-05 and FIND-06 to prevent recurrence.

Evidence: E-041

Risk Summary

Severity	Count	Findings	Max Score
Critical	4	FIND-02, FIND-05, FIND-06, FIND-07	100
High	2	FIND-03, FIND-04	81
Medium	1	FIND-01	40
Low	0	—	—

Note: Risk Score = Likelihood (1–10) × Impact (1–10). Scores: Critical ≥ 80 | High ≥ 40 | Medium ≥ 15 | Low < 15. Classification: Training Use Only | Author: Daniyal Ahmed | C11/26/EHIT/17331